

SIMATS ENGINEERING C02-ASSESSMENT TOOL-3

COURSE NAME: Theory of Computation COURSE CODE: CSA1304

COURSE OUTCOME COVERED

CO2: Design Finite Automata DFA, NFA, ϵ -NFA and demonstrate their equivalence for recognizing regular languages. (BL:3)

Assessment Tool Weightage: 20%

QUESTIONS: 4 Total Marks:20 Duration : 1 Hour

Scenario Based

Code of Conduct:

I GIRIJA B/ 192311044 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: GIRIJA B

SIMATS ENGINEERING ASSESSMENT TOOL

1. Given a DFA used for validating a specific PIN pattern `bbbc`, `bca`, `c`, `ca`, `caaaaaa` minimize the DFA and explain how state reduction improves performance and efficiency in embedded systems

Answer:

Step 1: Construct the DFA

The DFA accepts the strings:

- `bbbc`
- `bca`
- `c`
- `ca`
- `caaaaaa`

Initially, the DFA contains several states representing each character transition.

Step 2: DFA Minimization

Apply the DFA minimization algorithm:

- Remove **unreachable states**.
- Merge **equivalent states** (states with identical future behavior).
- Keep only the **essential states**.

The minimized DFA has **fewer states** while accepting exactly the same language.

Minimized DFA (State Transitions)

Current State	Input b	Input c	Input a
q0 (Start)	q1	q4	Dead
q1	q2	Dead	Dead
q2	q3	Dead	Dead
q3	Dead	q5 ✓	Dead
q4 ✓	Dead	Dead	q6 ✓
q6 ✓	Dead	Dead	q6
Dead	Dead	Dead	Dead

Accepting States: q4, q5, q6

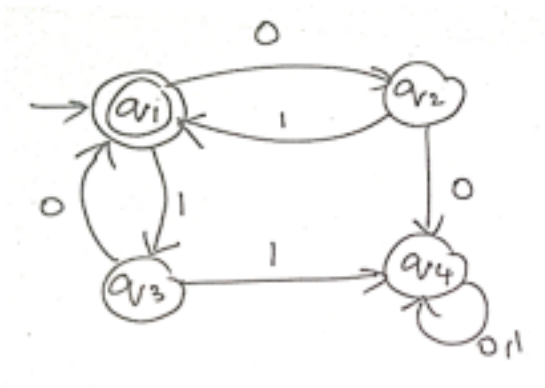
Benefits of State Reduction in Embedded Systems

- **Reduces memory usage** by storing fewer states.
- **Improves execution speed** due to fewer state transitions.
- **Consumes less power**, which is important in battery-powered embedded devices.
- **Simplifies hardware/software implementation**, making the design easier to maintain.
- **Increases overall system efficiency** while preserving the same functionality.

Conclusion

DFA minimization removes redundant states without changing the accepted language. This results in a smaller, faster, and more memory-efficient automaton, making it ideal for embedded systems with limited resources.

2.Demonstrate how a given finite automaton can be converted into a regular expression



The given finite automaton can be converted into a **Regular Expression (RE)** using the **State Elimination Method**.

Step 1: Identify the States

- Start state: q_1
- Final state: q_4
- Intermediate states: q_2, q_3

Step 2: Observe the Transitions

- $q_1 \rightarrow q_2$ on **0**
- $q_2 \rightarrow q_1$ on **1**
- $q_1 \rightarrow q_3$ on **1**
- $q_3 \rightarrow q_1$ on **0**
- $q_2 \rightarrow q_4$ on **0**
- $q_3 \rightarrow q_4$ on **1**
- q_4 has a self-loop on **0** and **1**, represented as **$(0+1)^*$**

Step 3: Eliminate Intermediate States

- Eliminate q_2 and q_3 one by one.

- Combine the transition labels while preserving all possible paths from the start state to the final state.

Paths from q_1 to q_4 are:

- $q_1 \rightarrow q_2 \rightarrow q_4 = 00$
- $q_1 \rightarrow q_3 \rightarrow q_4 = 11$

Since q_4 has a self-loop on **0** and **1**, append **(0+1)***.

Step 4: Regular Expression

$$(00 + 11)(0 + 1)^*$$

where:

- **+** denotes **union (OR)**.
- ***** denotes **Kleene star (zero or more occurrences)**.

Conclusion

The finite automaton is converted into the regular expression:

$$(00 + 11)(0 + 1)^*$$

This regular expression accepts the same language as the given finite automaton. Converting finite automata into regular expressions provides a compact representation of the language and is widely used in lexical analysis, compiler design, and pattern matching.

3.Consider the following grammar

$S \rightarrow aSc | T | U | e$

$T \rightarrow bTc | e$

U- $\rightarrow aUb|e$

A) What is the language defined by the grammar?

B) Give the derivation tree for the word aaabbc

C Grammar is Ambiguous if there exists two distinct derivations for a word. Is the above grammar ambiguous? Justify.

A) What is the language defined by the grammar?

The grammar generates the **union of three languages**:

1. $S \rightarrow aSc$
 - Generates equal numbers of **a's** followed by **c's**.
 - Language: $\{a^n c^n \mid n \geq 0\}$
2. $T \rightarrow bTc$
 - Generates equal numbers of **b's** followed by **c's**.
 - Language: $\{b^n c^n \mid n \geq 0\}$
3. $U \rightarrow aUb$
 - Generates equal numbers of **a's** followed by **b's**.
 - Language: $\{a^n b^n \mid n \geq 0\}$

Hence, the language is

$$L = \{a^n c^n \mid n \geq 0\} \cup \{b^n c^n \mid n \geq 0\} \cup \{a^n b^n \mid n \geq 0\}$$

including the empty string ϵ .

B) Give the derivation tree for the word aaabbc.

The string **aaabbc** has:

- $a = 3$
- $b = 2$
- $c = 1$

It **does not belong** to any language generated by the grammar because:

- It is **not** of the form $a^n c^n$.
- It is **not** of the form $b^n c^n$.
- It is **not** of the form $a^n b^n$.

Therefore:

aaabbc cannot be derived from the given grammar. Hence, no derivation tree exists.

C) Is the grammar ambiguous? Justify.

Yes, the grammar is ambiguous.

Justification:

The empty string ϵ can be derived in more than one way.

Derivation 1

$$S \Rightarrow \epsilon$$

Derivation 2

$$S \Rightarrow T \Rightarrow \epsilon$$

Derivation 3

$$S \Rightarrow U \Rightarrow \epsilon$$

Since the same string (ϵ) has **more than one distinct derivation**, the grammar is **ambiguous**.

Final Answer

A)

$$L = \{a^n c^n\} \cup \{b^n c^n\} \cup \{a^n b^n\}, n \geq 0$$

including ϵ .

B) The word aaabbc is not generated by the grammar; therefore, no derivation tree exists.

C) Yes, the grammar is ambiguous, because the empty string ϵ has multiple derivations:

- $S \Rightarrow \varepsilon$
- $S \Rightarrow T \Rightarrow \varepsilon$
- $S \Rightarrow U \Rightarrow \varepsilon$

4.I) Show that if L_1 and L_2 are regular then $(L_1 + L_2)$ also regular

ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify

4. (i) Show that if L_1 and L_2 are regular, then $(L_1 + L_2)$ is also regular.

Proof:

Assume L_1 and L_2 are regular languages.

- Since L_1 is regular, there exists a DFA M_1 that accepts L_1 .
- Since L_2 is regular, there exists a DFA M_2 that accepts L_2 .

Construct an NFA (or DFA) that accepts **either** L_1 **or** L_2 by combining the two automata.

Since regular languages are **closed under union**, the language

$$L_1 + L_2 = L_1 \cup L_2$$

is also regular.

Conclusion:

If L_1 and L_2 are regular, then $(L_1 + L_2)$ is also regular.

4. (ii) Is the language $\{a^p \mid p \text{ is prime}\}$ regular? Justify.

Answer: No, the language is not regular.

Justification (Using the Pumping Lemma):

Let

$$L = \{a^p \mid p \text{ is prime}\}$$

Assume L is regular.

Then, by the **Pumping Lemma**, there exists a pumping length n .

Choose a string

$$w = a^q$$

where q is a prime number and $q \geq n$.

Split the string as

$$w = xyz$$

such that:

- $|xy| \leq n$
- $|y| > 0$

Pumping y twice ($i = 2$) gives

$$xy^2z = a^{q+|y|}$$

The length $q+|y|$ is not necessarily prime. Hence, the pumped string may not belong to L .

This violates the Pumping Lemma.

Conclusion:

$\{a^p \mid p \text{ is prime}\}$ is not a regular language.